AI Chatbot — Project Overview

A general-purpose AI chat web app. Project decisions and reported progress from the conversation, as of 13 October 2026.

Updated hosting decision: deploy the frontend to Firebase Hosting. Keep Firebase Authentication and Firestore on Spark without enabling Firebase billing; deploy the backend to Render.

01 Project objective

Create a web interface where users can ask questions, request explanations, summarize or draft text, and explore ideas. Gemini generates the responses; the application provides the interface, account access and conversation management.

The first version is a general-purpose chatbot, with no specialized business domain agreed. The project also provides practical experience across frontend development, authentication, backend API integration, persistent history and deployment.

02 Technology stack

Layer	Selected technology	Responsibility
Frontend	React + TypeScript + Vite	Build and run the web interface.
Styling / typography	Tailwind CSS + Google Sans	Responsive layouts and consistent visual styling.
Backend	Node.js + Express + TypeScript	Receive chat requests, supply context and call Gemini.
AI service / model	Gemini Developer API / gemini-2.5-flash	Generate replies. Earlier model choice; confirm current availability and free-tier eligibility before integration.
AI SDK	@google/genai	Call Gemini from the backend.
Database	Firebase Cloud Firestore	Persist conversations and messages by user.
Authentication	Firebase Authentication	Google sign-in and user identity.
API key	GEMINI_API_KEY	Backend .env locally; Render environment variables after deployment.

The model and free tier are recorded project choices; live availability and quotas were not verified for this document. Google Sans was adopted during implementation.

The user experience

Three main pages share a warm white, sage and dark green design. Dialogs and modals do not count as additional pages.

03 Main pages

Page / route	Agreed role and elements
Sign-in /login	Google sign-in entry point, AI Chat branding and the reference design's sample conversation. The supplied setup guide also describes an in-memory Guest flow; real Google sign-in has not yet been confirmed.
Chat /chat /chat/:conversationId (proposed)	History sidebar, New chat, conversation messages and composer. Streaming replies show Stop. New and existing conversations use the same page. Rename and delete use dialogs in the agreed flow.
Settings /settings	Read-only account information, Light / Dark / System theme choices, Back to chat and Log out. Use the approved Settings reference image.

Frontend status: the user reports all three pages complete. Maintain the existing responsive approach across mobile, tablet, desktop and large screens; real service integration is a separate milestone.

04 From message to Gemini response

Step	Planned application behavior
1 Compose	The user opens a conversation and sends a text message from the React interface.
2 Request	The frontend sends the request to the Node.js / Express backend, running locally during development or on Render after deployment.
3 Context	The backend supplies the relevant conversation history with the new message. Firestore storage alone does not make Gemini remember the conversation.
4 Generate	The backend calls Gemini through @google/genai, using the API key held on the server.
5 Stream and retain	The reply streams back through the backend to the interface. The agreed end state stores conversation messages in Firestore so users can return and continue.

This is the planned end-to-end flow, not confirmed live behavior. Firestore read/write ownership and persistence timing remain implementation decisions. Guest history remains in memory under the supplied setup guide.

Deployment

Deployment targets for the frontend, backend, data and identity services.

05 Deployment plan

Component	During development	Planned deployment
Frontend	Vite on localhost; build to dist	Firebase Hosting (static frontend). Expected site: aichatbot-gemini.web.app.
Backend	Node.js / Express on localhost	Render Web Service; calls Gemini.
Data and identity	Connect to Firebase Cloud	Cloud Firestore + Firebase Auth on Spark. Firebase billing remains disabled.
Gemini secret	Backend .env	Environment variables on Render; never in frontend code.

Production flow: Firebase Hosting serves the frontend. Render calls Gemini and returns replies; Firestore retains signed-in users' history. This replaces the Vercel frontend plan.